

Escuela de Ingeniería Electrónica

EL5811 Verificación funcional de circuitos integrados

I Semestre 2026

Informe de Proyecto - grupo 1

Proyecto 2

Tomas Campos Uribe, Cruz Vargas Juan Pablo

Abstract

En este proyecto se desarrolló un ambiente de verificación funcional completo basado en la metodología UVM para el módulo `cfs_aligner`, un alineador de datos que recibe transferencias MD potencialmente no alineadas, valida su legalidad mediante la ecuación $\left(\frac{ALGN_DATA_WIDTH}{8} + offset\right) \bmod size = 0$, las reorganiza según los campos `CTRL.SIZE` y `CTRL.OFFSET`, y las emite como transferencias MD alineadas por la interfaz TX. El ambiente integra tres agentes UVM (APB, MD RX y MD TX), un modelo de referencia, un *scoreboard*, un modelo RAL, un secuenciador virtual y bloques de cobertura funcional con cuatro coverpoints y un cruce `size×offset`. Se ejecutaron seis escenarios de prueba: dos de alineamiento legal base (tráfico fijo y aleatorio), uno de estrés con 200 transacciones, y tres escenarios de retro-presión del TX (periódica, aleatoria y fuerte). En todos los casos el *scoreboard* reportó cero errores UVM y todos los bytes TX observados coincidieron con la predicción del modelo de referencia, incluyendo el escenario de estrés que produjo 323 matches correctos.

Palabras clave— UVM, verificación funcional, alineador de datos, AMBA APB, protocolo MD, agente UVM, scoreboard, modelo de referencia, RAL, cobertura funcional, SystemVerilog, `cfs_aligner`.

Link al repositorio: https://github.com/Tommycampos07/uvm_aligner_verification

Contents

1	Introducción	4
2	Objetivos	4
2.1	Objetivo General	4
2.2	Objetivos Específicos	4
3	El Dispositivo Bajo Prueba: <i>cfs_aligner</i>	5
3.1	Descripción General	5
3.2	Parámetros del Módulo	5
3.3	Señales de Entrada y Salida del DUT	6
4	Funcionamiento del Alineador de Datos	8
4.1	El Protocolo MD (<i>Memory Data</i>)	8
4.2	Legalidad de una Transferencia RX	8
4.3	Proceso de Alineamiento	9
4.4	Control de Flujo y FIFOs	9
4.5	Registros del DUT	10
4.6	Protocolo APB y Escenarios de Error	11
5	Ambiente de Verificación UVM	12
5.1	Arquitectura General	12
5.2	Flujo de Datos entre Componentes	13
5.3	Módulo <i>tb_top</i>	14
5.4	Módulos del Ambiente: Señales y Puertos	14
5.4.1	Ítem de Secuencia APB (<i>apb_item</i>)	14
5.4.2	Ítem de Secuencia MD (<i>md_item</i>)	14
5.4.3	Agente APB (<i>apb_agent</i>)	14
5.4.4	Agente MD RX (<i>md_rx_agent</i>)	15
5.4.5	Agente MD TX (<i>md_tx_agent</i>)	15
5.4.6	Modelo de Referencia (<i>aligner_reference_model</i>)	16
5.4.7	Scoreboard (<i>aligner_scoreboard</i>)	16
5.4.8	Cobertura Funcional (<i>aligner_coverage</i>)	17
5.4.9	Modelo RAL (<i>aligner_reg_block</i>)	18
5.4.10	Secuenciador Virtual (<i>aligner_virtualseq</i>)	18
6	Pruebas del Alineador	18
6.1	Plusargs Disponibles	18
6.2	Descripción de las Pruebas Diseñadas	19

6.2.1	Prueba de Alineamiento Legal Base (<code>aligner_legal_alignment_test</code>) . .	19
6.2.2	Caso Esquina: Retro-presión del TX (<code>aligner_tx_backpressure_test</code>) . .	22
6.2.3	Resumen Global de Pruebas	25
7	Conclusiones	25

1 Introducción

En el diseño de sistemas embebidos y de circuitos integrados, la verificación funcional es una etapa crítica que determina si un módulo de hardware se comporta de acuerdo con su especificación antes de ser llevado a fabricación o integración. A medida que la complejidad de los diseños aumenta, se vuelve imprescindible adoptar metodologías de verificación estructuradas y reutilizables.

El presente reporte documenta el desarrollo de un ambiente de verificación completo, basado en la metodología **UVM** (*Universal Verification Methodology*), para el módulo `cfs_aligner`. Este módulo recibe un flujo de datos no alineado a través de una interfaz MD RX (*Memory Data Receive*) y lo transforma en un flujo alineado que se emite a través de una interfaz MD TX (*Memory Data Transmit*), según la configuración establecida en sus registros internos accesibles mediante el protocolo AMBA 3 APB.

El ambiente implementado hace uso de agentes UVM activos y pasivos, un modelo de referencia, un *scoreboard*, un bloque de cobertura funcional y un modelo RAL (*Register Abstraction Layer*) para el acceso a registros. La prueba principal diseñada verifica el comportamiento de alineamiento legal del módulo bajo las siete configuraciones válidas de `CTRL.SIZE` y `CTRL.OFFSET` definidas en el *datasheet*.

2 Objetivos

2.1 Objetivo General

Desarrollar un ambiente de verificación funcional basado en la metodología UVM para el módulo `cfs_aligner`, capaz de estimular el DUT con tráfico legal e ilegal, comparar automáticamente la salida observada contra la salida esperada calculada por un modelo de referencia, y reportar resultados de cobertura funcional.

2.2 Objetivos Específicos

1. Analizar la especificación del módulo `cfs_aligner` a partir de su *datasheet*, identificando las interfaces, registros, parámetros y comportamientos funcionales relevantes para la verificación.
2. Diseñar e implementar los agentes UVM necesarios para estimular y monitorear las interfaces APB, MD RX y MD TX del DUT, incluyendo *drivers*, *monitors* y *sequencers*.
3. Construir un modelo de referencia que, a partir de los datos recibidos en la interfaz MD RX y la configuración activa en el registro CTRL, calcule de forma autónoma la salida esperada en la interfaz MD TX.
4. Implementar un *scoreboard* que compare de forma automática cada transacción observada en la

interfaz MD TX con su correspondiente predicción del modelo de referencia, reportando coincidencias y discrepancias.

5. Integrar un modelo RAL (*Register Abstraction Layer*) para el acceso estructurado a los registros del DUT a través del protocolo APB, utilizando el adaptador `aligner_apb_adapter`.
6. Desarrollar un grupo de cobertura funcional que registre las combinaciones de `size` y `offset` ejercitadas durante la simulación, verificando que todas las configuraciones legales sean estimuladas.
7. Diseñar y ejecutar la prueba de alineamiento legal (`aligner_legal_alignment_test`), que configura el DUT con una combinación válida de `CTRL.SIZE` y `CTRL.OFFSET`, envía tráfico MD RX legal y verifica la salida TX alineada mediante el *scoreboard*.

3 El Dispositivo Bajo Prueba: *cfs_aligner*

3.1 Descripción General

El módulo `cfs_aligner` es un alineador de datos que recibe un flujo de datos no alineado y lo convierte en un flujo alineado según su configuración. Su propósito es optimizar las escrituras en memoria realizando únicamente las escrituras más adecuadas para el tipo de memoria del sistema. Internamente, el módulo está compuesto por cinco bloques principales que conforman el flujo de datos de extremo a extremo:

1. **RX Controller:** Valida las transferencias entrantes y controla el retro-presión (*backpressure*) sobre la interfaz MD RX.
2. **RX FIFO:** Almacena temporalmente los datos recibidos hasta que el controlador de alineamiento esté listo.
3. **Controller:** Extrae los bytes válidos de los datos RX y los reposiciona en el bus de salida según `CTRL.SIZE` y `CTRL.OFFSET`.
4. **TX FIFO:** Almacena los datos ya alineados hasta que el controlador TX los envíe.
5. **TX Controller:** Saca los datos del TX FIFO y los transmite por la interfaz MD TX.

Adicionalmente, un bloque de **Register File** conecta la interfaz APB con todos los submódulos, exponiendo registros de control, estado e interrupciones.

3.2 Parámetros del Módulo

El módulo acepta dos parámetros de instanciación que definen el ancho del bus de datos y la profundidad de los FIFOs:

Table 1: Parámetros del módulo *cfs_aligner*

Parámetro	Valor por defecto	Descripción
ALGN_DATA_WIDTH	32	Ancho en bits del bus de datos. Debe ser potencia de 2; valor mínimo 8.
FIFO_DEPTH	8	Profundidad de los FIFOs RX y TX.

3.3 Señales de Entrada y Salida del DUT

El módulo expone tres grupos de señales: el reloj y reset globales, la interfaz APB para acceso a registros, y las dos interfaces MD para datos.

Señales Globales

Table 2: Señales globales del DUT

Señal	Ancho	Dir.	Descripción
clk	1	IN	Señal de reloj principal del módulo.
reset_n	1	IN	Reset activo en bajo.

Interfaz APB

Table 3: Señales de la interfaz APB del DUT

Señal	Ancho	Dir.	Descripción
psel	1	IN	Selección del esclavo APB.
penable	1	IN	Habilitación de fase de acceso APB.
pwrite	1	IN	Dirección de la transferencia (1=escritura).
paddr	16	IN	Dirección del registro. Los bits [1:0] se ignoran (acceso alineado a palabra).
pdata	32	IN	Dato a escribir.
pready	1	OUT	Indica que el esclavo completó la transferencia.
prdata	32	OUT	Dato leído del registro.
pslverr	1	OUT	Error del esclavo APB.

Interfaz MD RX

Table 4: Señales de la interfaz MD RX del DUT

Señal	Ancho	Dir.	Descripción
md_rx_valid	1	IN	Indica transferencia activa; debe mantenerse alto hasta que md_rx_ready suba.
md_rx_data	ALGN_DATA_WIDTH	IN	Dato no alineado. Estable mientras valid=1.
md_rx_offset	$\lceil \log_2(W/8) \rceil$	IN	Byte del bus desde el que inician los datos válidos.
md_rx_size	$\lceil \log_2(W/8) \rceil + 1$	IN	Número de bytes válidos en el bus ($\neq 0$).
md_rx_ready	1	OUT	<i>Handshake</i> : el DUT acepta la transferencia.
md_rx_err	1	OUT	Error en la transferencia (sólo válido en valid && ready).

$$W = \text{ALGN_DATA_WIDTH}$$

Interfaz MD TX

Table 5: Señales de la interfaz MD TX del DUT

Señal	Ancho	Dir.	Descripción
md_tx_valid	1	OUT	Indica dato alineado disponible.
md_tx_data	ALGN_DATA_WIDTH	OUT	Dato alineado.
md_tx_offset	$\lceil \log_2(W/8) \rceil$	OUT	Posición del primer byte válido en el bus TX (igual a CTRL.OFFSET).
md_tx_size	$\lceil \log_2(W/8) \rceil + 1$	OUT	Número de bytes válidos en el bus TX (igual a CTRL.SIZE).
md_tx_ready	1	IN	El receptor externo acepta la transferencia TX.
md_tx_err	1	IN	Error del receptor (sólo válido en valid && ready).
irq	1	OUT	OR de todas las interrupciones habilitadas.

4 Funcionamiento del Alineador de Datos

4.1 El Protocolo MD (*Memory Data*)

Ambas interfaces, RX y TX, utilizan el mismo protocolo MD. Las reglas fundamentales son:

1. Una transferencia inicia cuando valid sube a 1.
2. La transferencia termina cuando simultáneamente valid = 1 y ready = 1 (handshake).
3. Una vez que valid se afirma, debe mantenerse alto hasta que ready suba.
4. Las señales data, offset y size deben permanecer constantes desde el inicio hasta el final de la transferencia.
5. La señal err sólo es válida durante el ciclo de handshake.

4.2 Legalidad de una Transferencia RX

No toda combinación de offset y size es legal. Una transferencia MD RX se considera legal si y sólo si se cumplen ambas condiciones:

$$\text{size} \neq 0 \quad \text{y} \quad \left(\frac{\text{ALGN_DATA_WIDTH}}{8} + \text{offset} \right) \bmod \text{size} = 0 \quad (1)$$

Con ALGN_DATA_WIDTH = 32 (4 bytes por palabra), las siete combinaciones legales son las que se muestran en la Tabla 6.

Table 6: Combinaciones legales de CTRL.SIZE y CTRL.OFFSET

CTRL.SIZE	CTRL.OFFSET	Descripción
1	0	1 byte en posición 0
1	1	1 byte en posición 1
1	2	1 byte en posición 2
1	3	1 byte en posición 3
2	0	2 bytes en posición 0
2	2	2 bytes en posición 2
4	0	Palabra completa de 4 bytes

Si la ecuación no se satisface, el RX Controller responde con `md_rx_err = 1`, la transferencia es descartada y el contador `STATUS.CNT_DROP` se incrementa (sin superar su valor máximo de 255).

4.3 Proceso de Alineamiento

El Controller extrae los bytes válidos del dato RX y los reposiciona en el bus de salida TX. El algoritmo, derivado directamente del RTL (`cfs_ctrl.v`), opera así:

1. Los bytes válidos del dato RX se extraen empezando en la posición `rx_offset`, tomando `rx_size` bytes en total.
2. Esos bytes se acumulan en una cola interna.
3. Cuando la cola contiene al menos `CTRL.SIZE` bytes, se forma un paquete TX: los primeros `CTRL.SIZE` bytes de la cola se colocan en la posición `CTRL.OFFSET` del bus de salida.
4. La salida TX siempre presenta `md_tx_size = CTRL.SIZE` y `md_tx_offset = CTRL.OFFSET`, independientemente de la configuración de la entrada RX.

La fórmula de posicionamiento es:

$$\text{mask} = (1 \ll (\text{CTRL.SIZE} \times 8)) - 1 \quad (2)$$

$$\text{extracted} = \text{mask} \& (\text{rx_data} \gg (\text{src_byte} \times 8)) \quad (3)$$

$$\text{tx_data} = \text{extracted} \ll (\text{CTRL.OFFSET} \times 8) \quad (4)$$

donde `src_byte = rx_offset + bytes_procesados`.

4.4 Control de Flujo y FIFOs

El RX Controller aplica retro-presión desafiando `md_rx_ready` cuando el RX FIFO está lleno. El nivel de ocupación actual de cada FIFO es visible en el registro `STATUS`: `STATUS.RX_LVL` (bits [11:8])

y STATUS.TX_LVL (bits [19:16]).

Cuando el TX FIFO está lleno, el Controller detiene el procesamiento hasta que haya espacio disponible.

4.5 Registros del DUT

El acceso a los registros se realiza mediante el protocolo AMBA 3 APB. El módulo tiene cuatro registros mapeados en las siguientes direcciones:

Table 7: Mapa de registros del cfs_aligner

Nemónico	Dirección	Nombre
CTRL	0x0000	Control Register
STATUS	0x000C	Status Register
IRQEN	0x00F0	Interrupt Requests Enable Register
IRQ	0x00F4	Interrupt Requests Register

Registro CTRL (0x0000)

Contiene los campos de configuración del alineamiento. Sus campos principales son:

Table 8: Campos del registro CTRL

Campo	Bits	Tamaño	Acceso	Reset	Descripción
SIZE	[2:0]	3	RW	1	Tamaño en bytes de los datos alineados de salida. SIZE=0 es ilegal.
OFFSET	[9:8]	2	RW	0	Desplazamiento en bytes de los datos de salida.
CLR	[16]	1	WO	0	Escrito con 1 limpia STATUS.CNT_DROP. Lectura siempre retorna 0.

Escribir una combinación ilegal de SIZE y OFFSET retorna `pslverr = 1` y el registro no se modifica. El valor de reset es 0x0000_0001 (SIZE=1, OFFSET=0).

Registro STATUS (0x000C)

Registro de sólo lectura (RO). Contiene información de estado en tiempo real:

Table 9: Campos del registro STATUS

Campo	Bits	Acceso	Reset	Descripción
CNT_DROP	[7:0]	RO	0	Contador de transferencias ilegales descartadas. Satura en 255 sin desbordarse.
RX_LVL	[11:8]	RO	0	Nivel de ocupación del RX FIFO.
TX_LVL	[19:16]	RO	0	Nivel de ocupación del TX FIFO.

Registro IRQEN (0x00F0) e IRQ (0x00F4)

El registro IRQEN habilita individualmente cada interrupción (acceso RW, reset 0x1F). El registro IRQ almacena el estado de cada interrupción con semántica *Write-1-Clear* (W1C). Ambos registros comparten la misma distribución de campos:

Table 10: Campos de los registros IRQEN e IRQ

Campo	Bit	Evento
RX_FIFO_EMPTY	0	El RX FIFO se vació (comportamiento pegajoso).
RX_FIFO_FULL	1	El RX FIFO se llenó (comportamiento pegajoso).
TX_FIFO_EMPTY	2	El TX FIFO se vació (comportamiento pegajoso).
TX_FIFO_FULL	3	El TX FIFO se llenó (comportamiento pegajoso).
MAX_DROP	4	CNT_DROP alcanzó su valor máximo (comportamiento pegajoso).

Todas las interrupciones tienen comportamiento *sticky*: una vez activas permanecen en 1 hasta que el software las limpie mediante escritura de 1 en el campo correspondiente del registro IRQ.

4.6 Protocolo APB y Escenarios de Error

El DUT implementa el estándar AMBA 3 APB. El acceso se realiza en dos fases: *Setup* (un ciclo con `psel=1`, `penable=0`) y *Access* (`psel=1`, `penable=1`) con `pready` indicando la finalización. Se permiten hasta 5 ciclos de espera (*wait states*).

El DUT responde con `pslverr = 1` en los siguientes escenarios:

- Acceso a una dirección no mapeada.

- Escritura al registro STATUS (sólo lectura).
- Escritura a CTRL con una combinación ilegal de CTRL.SIZE y CTRL.OFFSET.

5 Ambiente de Verificación UVM

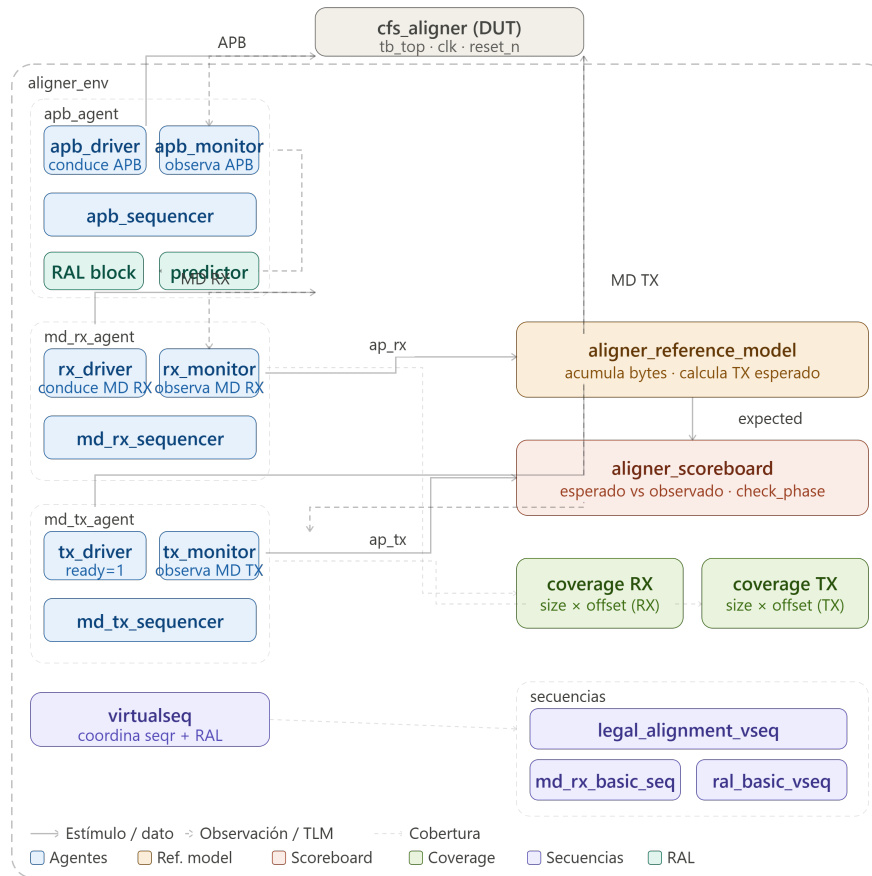


Figure 1: Diagrama general del ambiente de verificación

5.1 Arquitectura General

El ambiente sigue una arquitectura UVM estándar organizada en capas. El `tb_top` es el módulo SystemVerilog no-UVM que instancia el DUT, genera el reloj, aplica el reset e inyecta las interfaces virtuales en el `uvm_config_db`. Desde ahí, la jerarquía UVM es:

- **Test** (`aligner_legal_alignment_test`)
 - **Environment** (`aligner_env`)

- * `apb_agent` (activo): agente APB
- * `md_rx_agent` (activo): agente MD RX
- * `md_tx_agent` (activo): agente MD TX
- * `aligner_reference_model`: modelo de referencia
- * `aligner_scoreboard`: comparador
- * `aligner_coverage` ×2: cobertura RX y TX
- * `aligner_reg_block`: modelo RAL
- * `aligner_apb_adapter`: adaptador RAL↔APB
- * `uvm_reg_predictor`: predictor del modelo RAL
- * `aligner_virtalseq`: secuenciador virtual

5.2 Flujo de Datos entre Componentes

El flujo de información a través del ambiente sigue el siguiente camino:

1. El test lanza la secuencia virtual `legal_alignment_vseq` sobre el secuenciador virtual.
2. La secuencia virtual usa el modelo RAL para escribir y leer el registro CTRL, configurando SIZE y OFFSET.
3. Luego lanza `md_rx_basic_seq` sobre el secuenciador del `md_rx_agent`, que genera ítems `md_item` con datos, offset y size legales.
4. El `md_rx_driver` toma cada ítem y conduce las señales de la interfaz MD RX del DUT.
5. El `md_rx_monitor` observa pasivamente el bus MD RX y publica cada transacción completada en su *analysis port*.
6. El `aligner_reference_model` recibe cada ítem RX observado, extrae los bytes válidos, los acumula, y cuando tiene suficientes para completar un paquete TX (según la configuración de SIZE), genera el ítem esperado y lo publica en su propio *analysis port*.
7. Cuando el DUT produce una salida alineada, el `md_tx_monitor` la captura y la publica en su *analysis port*.
8. El `aligner_scoreboard` recibe tanto el ítem esperado (del modelo de referencia) como el ítem observado (del monitor TX) y los compara campo por campo.
9. El monitor APB (`apb_monitor`) alimenta al predictor del modelo RAL, manteniéndolo sincronizado con el estado real del DUT.
10. Los monitores RX y TX también alimentan a sus respectivos bloques de cobertura (`aligner_coverage`).

5.3 Módulo `tb_top`

El *top* del testbench es un módulo SystemVerilog convencional que actúa como puente entre el hardware y el mundo UVM. Sus responsabilidades son:

- Generación de reloj: oscilador con período de 10 ns (100 MHz).
- Aplicación de reset: `reset_n` se mantiene en bajo durante 5 ciclos y luego se libera.
- Instanciación de interfaces: `apb_if`, `md_if` para RX y TX.
- Instanciación y conexión del DUT `cfs_aligner`.
- Registro de las interfaces virtuales en `uvm_config_db`.
- Llamada a `run_test()` para que UVM seleccione el test mediante el *plusarg* `+UVM_TESTNAME`.

5.4 Módulos del Ambiente: Señales y Puertos

5.4.1 Ítem de Secuencia APB (`apb_item`)

Representa una transacción APB completa. Sus campos son:

Table 11: Campos del `apb_item`

Campo	Tipo	Descripción
<code>pwrite</code>	<code>rand bit</code>	Dirección (1=escritura, 0=lectura).
<code>paddr</code>	<code>rand bit[15:0]</code>	Dirección del registro.
<code>pdata</code>	<code>rand bit[31:0]</code>	Dato a escribir.
<code>prdata</code>	<code>bit[31:0]</code>	Dato leído (relleno por el driver).
<code>pslverr</code>	<code>bit</code>	Error del esclavo (relleno por el driver).

5.4.2 Ítem de Secuencia MD (`md_item`)

Representa una transacción MD (usada tanto para RX como para TX). Sus campos son:

5.4.3 Agente APB (`apb_agent`)

Agrupar el driver, monitor y sequencer para la interfaz APB. Opera en modo activo (`UVM_ACTIVE`).

El monitor APB expone un *analysis port* (`ap`) del tipo `uvm_analysis_port#(apb_item)`, conectado al predictor del modelo RAL en el environment.

Table 12: Campos del md_item

Campo	Tipo	Descripción
data	rand bit[31:0]	Palabra de datos (32 bits).
offset	rand bit[1:0]	Byte de inicio del dato válido en el bus.
size	rand bit[2:0]	Número de bytes válidos.
err	bit	Indicador de error de transferencia.

Table 13: Componentes y puertos del apb_agent

Sub-componente	Tipo	Función
seqr	uvm_sequencer#(apb_item)	Canal entre secuencias y driver.
drv	apb_driver	Conduce señales APB en la interfaz.
mon	apb_monitor	Observa y publica transacciones APB.

5.4.4 Agente MD RX (md_rx_agent)

Controla el lado maestro de la interfaz MD RX, inyectando tráfico hacia el DUT.

Table 14: Componentes y puertos del md_rx_agent

Sub-componente	Tipo	Función
seqr	uvm_sequencer#(md_item)	Canal entre secuencias y driver.
drv	md_rx_driver	Conduce valid, data, offset, size.
mon	md_rx_monitor	Observa handshakes RX completados.

El monitor RX expone un *analysis port* (ap) conectado al modelo de referencia y al bloque de cobertura RX.

5.4.5 Agente MD TX (md_tx_agent)

Actúa como esclavo reactivo en la interfaz MD TX: mantiene `ready = 1` de forma permanente para no bloquear el flujo del DUT y el monitor observa la salida.

Table 15: Componentes y puertos del md_tx_agent

Sub-componente	Tipo	Función
seqr	uvm_sequencer#(md_item)	Canal entre secuencias y driver.
drv	md_tx_driver	Mantiene md_tx_ready = 1 y err = 0.
mon	md_tx_monitor	Observa y publica transacciones TX completadas.

El monitor TX expone un *analysis port* (ap) conectado al *scoreboard* (puerto de observaciones reales) y al bloque de cobertura TX.

5.4.6 Modelo de Referencia (aligner_reference_model)

Recibe cada ítem RX observado por el monitor RX y calcula la salida TX esperada. Su interfaz de comunicación TLM es:

Table 16: Puertos TLM del modelo de referencia

Puerto	Tipo	Rol
rx_export	uvm_analysis_imp_rx	Recibe ítems del monitor RX.
expected_ap	uvm_analysis_port#(md_item)	Publica ítems TX esperados al scoreboard.

El modelo mantiene una cola interna de bytes (byte_q). Cuando la cola acumula al menos `cfg_tx_size` bytes, forma un ítem TX esperado y lo publica. Sus parámetros internos `cfg_tx_size` y `cfg_tx_offset` se actualizan llamando al método `configure_output(size, offset)`, invocado por la secuencia virtual tras cada escritura exitosa al registro CTRL.

5.4.7 Scoreboard (aligner_scoreboard)

Recibe y compara las transacciones esperadas (del modelo de referencia) con las observadas (del monitor TX):

Cada vez que llegan un ítem esperado y uno observado, el scoreboard compara los campos `data`, `offset`, `size` y `err`. Si hay discrepancia, emite un `uvm_error`. Al final de la simulación (`check_phase`), verifica que ambas colas queden vacías.

Table 17: Puertos TLM del scoreboard

Puerto	Tipo	Rol
expected_export	uvm_analysis_imp_expected	Recibe ítems esperados.
actual_export	uvm_analysis_imp_actual	Recibe ítems observados.

5.4.8 Cobertura Funcional (aligner_coverage)

Se instancia dos veces en el environment: una para el tráfico RX y otra para TX. Cada instancia es un `uvm_subscriber#(md_item)` con un *covergroup* `md_cg` que mide cuatro coverpoints y un cruce:

Table 18: Coverpoints del *covergroup* `md_cg`

Coverpoint	Descripción y bins
cp_size	Valores de size: <code>byte_access={1}</code> , <code>halfword_access={2}</code> , <code>word_access={4}</code> . Define <code>illegal_bins zero_size={0}</code> para marcar explícitamente el valor ilegal.
cp_offset	Valores de offset: <code>offset_0={0}</code> , <code>offset_1={1}</code> , <code>offset_2={2}</code> , <code>offset_3={3}</code> .
cp_err	Estado del campo err: <code>no_error={0}</code> , <code>error={1}</code> . Permite verificar que ambas condiciones — transferencia aceptada y transferencia con error — son ejercitadas.
cp_data_lsb	Rango del byte menos significativo de <code>data[7:0]</code> : <code>low_values=[0x00:0x3F]</code> , <code>mid_values=[0x40:0xBF]</code> , <code>high_values=[0xC0:0xFF]</code> . Verifica diversidad en el valor de los datos transmitidos.
cross_size_offset	Producto cruzado de <code>cp_size × cp_offset</code> . Registra las combinaciones efectivamente ejercitadas y permite identificar cuáles de los doce cruces posibles son legales según el DUT.

El componente opera exclusivamente como observador: no conduce señales ni genera estímulos. Su método `write()` es invocado automáticamente por el mecanismo TLM cada vez que un monitor publica un ítem, momento en el que se llama `md_cg.sample()` y se imprime el porcentaje de cobertura acumulada mediante `md_cg.get_inst_coverage()`. Al finalizar la simulación, `report_phase` emite el porcentaje de cobertura final de cada instancia.

5.4.9 Modelo RAL (`aligner_reg_block`)

Proporciona una abstracción orientada a objetos de los registros del DUT. Contiene cuatro instancias de `aligner_reg_32` (una por registro) mapeadas en el bus APB mediante `apb_map`:

Table 19: Registros en el modelo RAL

Registro RAL	Dirección	Acceso RAL
CTRL	0x0000	RW
STATUS	0x000C	RO
IRQEN	0x00F0	RW
IRQ	0x00F4	RW

El adaptador `aligner_apb_adapter` traduce entre las operaciones de registro UVM (`uvm_reg_bus_op`) y los ítems APB concretos. El predictor `uvm_reg_predictor#(apb_item)` escucha el monitor APB y actualiza el modelo RAL sin necesitar leer el DUT.

5.4.10 Secuenciador Virtual (`aligner_virtualseq`)

Es el punto de coordinación de todas las secuencias. Extiende `uvm_sequencer` y mantiene referencias a:

- `apb_seqr`: secuenciador del agente APB.
- `md_rx_seqr`: secuenciador del agente MD RX.
- `md_tx_seqr`: secuenciador del agente MD TX.
- `ral_model`: referencia al modelo RAL para acceso directo desde las secuencias virtuales mediante `'uvm_declare_p_sequencer`.

6 Pruebas del Alineador

6.1 Plusargs Disponibles

El testbench acepta los siguientes *plusargs* de línea de comandos que permiten modificar el comportamiento de la simulación sin recompilar:

Ejemplo de invocación con VCS:

```
./simv +UVM_TESTNAME=aligner_legal_alignment_test \
      +UVM_VERBOSITY=UVM_MEDIUM \
      +ntb_random_seed=42 \
      +NUM_ITEMS=20 \
```

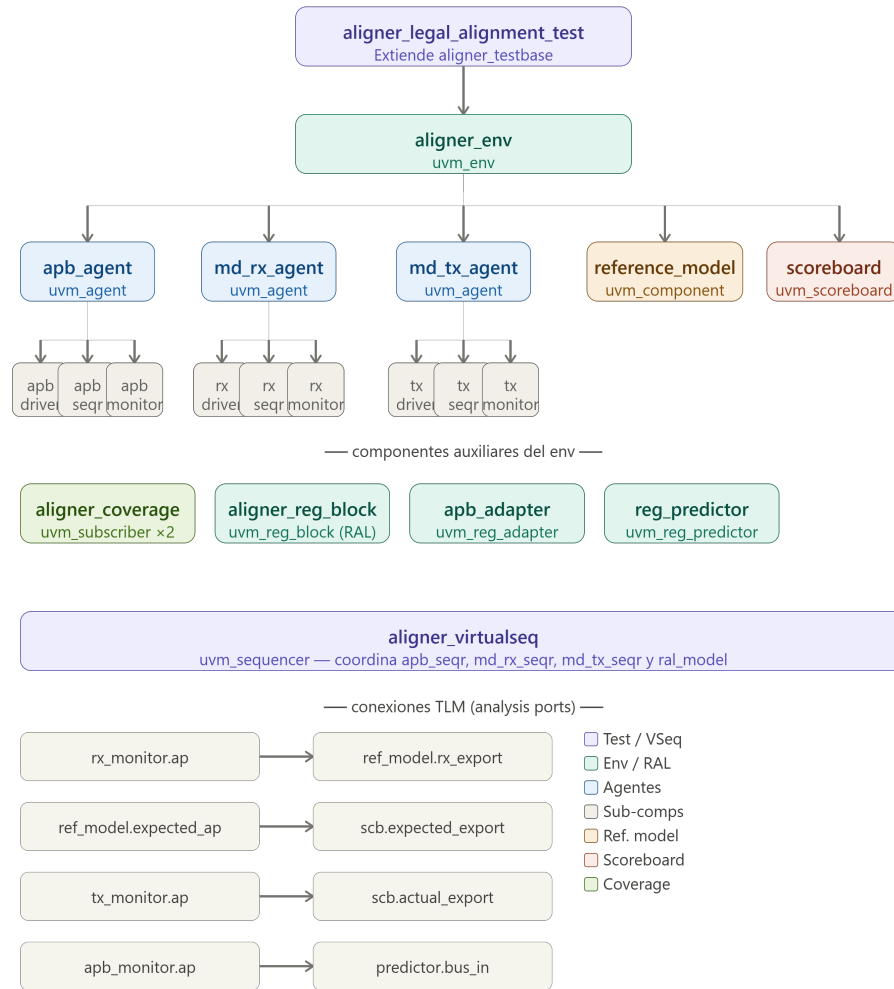


Figure 2: Diagrama de la prueba base del aligner

```
+FIXED_TRAFFIC=0
```

6.2 Descripción de las Pruebas Diseñadas

Las pruebas desarrolladas se dividen en dos grupos: la prueba de alineamiento legal base y los casos esquinosa. Cada una se ejecuta seleccionando el test correspondiente mediante el plusarg `+UVM_TESTNAME`.

6.2.1 Prueba de Alineamiento Legal Base (aligner_legal_alignment_test)

Esta prueba constituye la verificación funcional principal del DUT. Su objetivo es confirmar que el módulo recibe tráfico MD RX válido, lo procesa correctamente y emite transferencias MD TX alineadas que coinciden con la predicción del modelo de referencia. Opera en dos modos seleccionables mediante plusarg:

Table 20: Plusargs del testbench

Plusarg	Default	Descripción
+UVM_TESTNAME	—	Nombre del test UVM a ejecutar.
+UVM_VERBOSITY	UVM_LOW	Nivel de detalle del log UVM.
+ntb_random_seed	1	Semilla del generador aleatorio.
+NUM_ITEMS	7	Número de ítems MD RX en md_rx_basic_seq cuando FIXED_TRAFFIC=0.
+FIXED_TRAFFIC	1	1=tráfico fijo predefinido; 0=tráfico aleatorio legal.
+TX_READY_MODE	0	Modo del driver MD TX: 0=siempre ready, 1=periódico, 2=aleatorio.
+TX_READY_HIGH_CYCLES	5	Ciclos con ready=1 en modo periódico.
+TX_READY_LOW_CYCLES	3	Ciclos con ready=0 en modo periódico.
+TX_READY_RANDOM_PCT	70	Porcentaje de ciclos con ready=1 en modo aleatorio.
+DRAIN_TIME_NS	200	Tiempo de espera en ns al final de la secuencia virtual para vaciar el TX FIFO antes de terminar la simulación.

Modo fijo (+FIXED_TRAFFIC=1) La secuencia `legal_alignment_vseq` configura el DUT con `CTRL.SIZE=1` y `CTRL.OFFSET=0`, y lanza `md_rx_basic_seq` con los siete pares legales predefinidos del testplan. El modelo de referencia predice los 12 bytes de salida TX esperados. Al finalizar el tráfico, la secuencia lee el registro `STATUS` y verifica que `CNT_DROP = 0x00000000`.

Resultado obtenido: el scoreboard reportó **12 matches**, con `UVM_ERROR: 0` y `UVM_FATAL: 0`. La secuencia esperada fue DD CC BB AA 44 33 66 55 EF BA 34 87, que coincidió exactamente con la salida del DUT. El registro `STATUS` al final fue `0x00000000`.

Modo aleatorio (+FIXED_TRAFFIC=0) En este modo `md_rx_basic_seq` genera `NUM_ITEMS` transacciones aleatorias cuyo `offset` y `size` satisfacen la ecuación de legalidad. El número de matches en el scoreboard no es necesariamente igual a `NUM_ITEMS` porque el número de bytes TX generados depende de los valores de `size` de cada transacción RX.

Resultado obtenido con `+NUM_ITEMS=30`, `+ntb_random_seed=123`: el scoreboard reportó **42 matches**, con `UVM_ERROR: 0` y `UVM_FATAL: 0`. El valor de `STATUS` al finalizar fue `0x00000000`. El número de matches (42) es mayor que el de transacciones (30) porque varias transacciones con `size=2` o

size=4 generan más de un byte TX por cada ítem RX.

```

CAMPOS_TOMAS@redhat003:/mnt/vol_NFS_rh003/Est_Veri_1_2026/CAMPOS_TOMAS/uv_m_aligner_verification/sim
(CAMPOS_TOMAS@redhat003 sim)$ grep -i "Sequence configuration\\|UVM_ERROR\\|UVM_FATAL\\|SCB_MATCH\\|MISMATCH\\|STATUS" sim_fixed.log
UVM_INFO ../uv_m_seqs/md_rx_basic_seq.sv(24) @ 175000: uvm_test_top.env.md_rx_agnt.seqr@md_seq [MD_RX_BASIC_SEQ] Sequence configuration: NUM_ITEMS=7 FIXED_TRAFFIC=1
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 225000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000dd offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 235000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000cc offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 245000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000bb offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 255000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000aa offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 265000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000044 offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 275000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000033 offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 285000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000066 offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 295000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000055 offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 305000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000ef offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 315000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000ba offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 325000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000034 offset=0 size=1 err=0
UVM_INFO ../uv_m/env/aligner_scoreboard.sv(92) @ 345000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000087 offset=0 size=1 err=0
UVM_INFO ../uv_m_seqs/legal_alignment_vseq.sv(83) @ 515000: uvm_test_top.env.vseqr@legal_seq [LEGAL_ALIGNMENT_VSEQ] Reading STATUS register
UVM_INFO ../uv_m_seqs/legal_alignment_vseq.sv(99) @ 555000: uvm_test_top.env.vseqr@legal_seq [LEGAL_ALIGNMENT_VSEQ] STATUS value after traffic = 0x00000000
Number of demoted UVM_FATAL reports : 0
Number of demoted UVM_ERROR reports : 0
Number of caught UVM_FATAL reports : 0
Number of caught UVM_ERROR reports : 0
UVM_ERROR : 0
UVM_FATAL : 0
(SCB_MATCH) 12

```

Figure 3: Corrida de alineamiento legal base fijo

Table 21: Resumen de resultados de la prueba base

Modo	NUM_ITEMS	SCB_MATCH	UVM_ERROR	STATUS
Fijo (fixed)	7	12	0	0x00000000
Aleatorio (random)	30	42	0	0x00000000

```
[CAMPOS TOMAS@redhat003 sim]$ grep -i "Sequence configuration\|UVM_ERROR\|UVM_FATAL\|SCB_MATCH\|MISMATCH\|STATUS" sim_random.log
UVM_INFO ../uvm/seqs/md_rx_basic_seq.sv(24) @ 175000: uvm_test_top.env.md_rx_agnt.sqr88md_seq [MD_RX_BASIC_SEQ] Sequence configuration: NUM_ITEMS=30 FIXED_TRAFFIC=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 225000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x0000006d offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 245000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x0000006e offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 255000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000b9 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 265000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000a3 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 275000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000077 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 285000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000ac offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 305000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x0000008f offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 325000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x0000001d offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 345000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x0000002d offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 365000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000076 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 385000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000046 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 405000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000aa offset=0 size=1 err=0

CAMPOS TOMAS@redhat003:/mnt/vol_NFS_rh003/Est_Verif_1_2026/CAMPOS TOMAS/uvm_aligner_verification/sim
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 635000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000069 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 645000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000e2 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 655000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000030 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 665000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x0000009c offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 675000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x0000009e offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 685000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000d6 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 695000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000a8 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 705000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000074 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 725000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x0000004c offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 745000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000e9 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 765000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000035 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 785000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000bc offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 805000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x00000088 offset=0 size=1 err=0
UVM_INFO ../uvm/env/aligner_scoreboard.sv(92) @ 815000: uvm_test_top.env.scb [SCB_MATCH] Match TX: data=0x000000d1 offset=0 size=1 err=0
UVM_INFO ../uvm/seqs/legal_alignment_vseq.sv(83) @ 975000: uvm_test_top.env.vseq [LEGAL_ALIGNMENT_VSEQ] Reading STATUS register
UVM_INFO ../uvm/seqs/legal_alignment_vseq.sv(99) @ 1015000: uvm_test_top.env.vseq [LEGAL_ALIGNMENT_VSEQ] STATUS value after traffic = 0x00000000
Number of demoted UVM_FATAL reports : 0
Number of demoted UVM_ERROR reports : 0
Number of caught UVM_FATAL reports : 0
Number of caught UVM_ERROR reports : 0
UVM_ERROR : 0
UVM_FATAL : 0
[SCB_MATCH] 42
[CAMPOS TOMAS@redhat003 sim]$
```

Figure 4: Corrida de alineamiento legal base aleatorio

6.2.2 Caso Esquina: Retro-presión del TX (aligner_tx_backpressure_test)

Clases involucradas: aligner_tx_backpressure_test → legal_alignment_vseq → md_rx_basic_seq + md_tx_driver en modo configurable.

Esta prueba extiende aligner_testbase y lanza la misma secuencia virtual que la prueba base (legal_alignment_vseq), pero con el driver MD TX configurado en modo de retro-presión. El objetivo es verificar que el DUT no pierde datos cuando el receptor en la interfaz TX no puede aceptarlos inmediatamente: cuando md_tx_ready=0 el DUT debe mantener md_tx_valid=1 y los

```

CAMPOS_TOMAS@redhat003:/mnt/vol_NFS_rh003/Est_Veri_I_2026/CAMPOS_TOMAS/uvr_aligner_verification/sim
t.seqr@md_seq [MD_RX_BASIC_SEQ] Sequence configuration: NUM_ITEMS=7 FIXED_TRAFFIC
IC=1
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 225000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x000000dd offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 235000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x000000cc offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 245000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x000000bb offset=0 size=1 err=0
UVM INFO ../uvr/seqs/legal_alignment_vseq.sv(85) @ 315000: uvm_test_top.env.vseq
r@legal_seq [LEGAL_ALIGNMENT_VSEQ] Waiting 3000 ns for TX drain
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 335000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x000000aa offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 345000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x00000044 offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 355000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x00000033 offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 365000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x00000066 offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 375000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x00000055 offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 465000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x000000ef offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 475000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x000000ba offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 485000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x00000034 offset=0 size=1 err=0
UVM INFO ../uvr/env/aligner_scoreboard.sv(92) @ 495000: uvm_test_top.env.scb [SC
B_MATCH] Match TX: data=0x00000087 offset=0 size=1 err=0
UVM INFO ../uvr/seqs/legal_alignment_vseq.sv(91) @ 3315000: uvm_test_top.env.vse
qr@legal_seq [LEGAL_ALIGNMENT_VSEQ] Reading STATUS register
UVM INFO ../uvr/seqs/legal_alignment_vseq.sv(107) @ 3355000: uvm_test_top.env.vse
qr@legal_seq [LEGAL_ALIGNMENT_VSEQ] STATUS value after traffic = 0x00000000
Number of demoted UVM_FATAL reports : 0
Number of demoted UVM_ERROR reports : 0
Number of caught UVM_FATAL reports : 0
Number of caught UVM_ERROR reports : 0
UVM_ERROR : 0
UVM_FATAL : 0
[SCB_MATCH] 12
(CAMPOS_TOMAS@redhat003 sim)

```

Figure 5: Corrida del caso de esquina retro-presión periódica del TX.

datos estables hasta que ready vuelva a 1.

El driver soporta tres modos controlados por plusargs:

Se ejecutaron cuatro escenarios con este test:

Escenario 1 — Backpressure periódico fijo

```

./simv +UVM_TESTNAME=aligner_tx_backpressure_test \
      +UVM_VERBOSITY=UVM_LOW +FIXED_TRAFFIC=1 \
      +TX_READY_MODE=1 +TX_READY_HIGH_CYCLES=2 \
      +TX_READY_LOW_CYCLES=3 -l sim_bp_periodic_fixed.log

```

El driver alterna 2 ciclos con ready=1 y 3 ciclos con ready=0 de forma indefinida. Se enviaron las 7 transferencias fijas conocidas.

Resultado: SCB_MATCH: 12, UVM_ERROR: 0, UVM_FATAL: 0, STATUS = 0x00000000.

Escenario 2 — Backpressure aleatorio

```

./simv +UVM_TESTNAME=aligner_tx_backpressure_test \
      +UVM_VERBOSITY=UVM_LOW +FIXED_TRAFFIC=0 \
      +NUM_ITEMS=30 +TX_READY_MODE=2 \
      +TX_READY_RANDOM_PCT=60 +ntb_random_seed=321 \
      -l sim_bp_random.log

```

Table 22: Plusargs del driver MD TX para la prueba de backpressure

Plusarg	Default	Descripción
+TX_READY_MODE	0	0=siempre ready, 1=periódico, 2=aleatorio.
+TX_READY_HIGH_CYCLES	5	Ciclos con ready=1 en modo periódico.
+TX_READY_LOW_CYCLES	3	Ciclos con ready=0 en modo periódico.
+TX_READY_RANDOM_PCT	70	Porcentaje de ciclos con ready=1 en modo aleatorio.
+DRAIN_TIME_NS	–	Tiempo de espera en ns al final del test para vaciar el TX FIFO.

El driver decide ciclo a ciclo si ready es 1 o 0 con una probabilidad del 60 % y 40 % respectivamente. Se generaron 30 transacciones RX aleatorias legales. Cuando ready=0 el DUT no puede completar la transferencia TX aunque tenga valid=1, por lo que debe conservar el dato hasta que ready vuelva a 1.

Resultado: SCB_MATCH: 44, UVM_ERROR: 0, UVM_FATAL: 0, STATUS = 0x00000000. Los 44 matches representan la suma total de bytes válidos generados por las 30 transacciones RX, cuyo size varía entre 1, 2 y 4.

Escenario 3 — Backpressure fuerte fijo

```
./simv +UVM_TESTNAME=aligner_tx_backpressure_test \
      +UVM_VERBOSITY=UVM_LOW +FIXED_TRAFFIC=1 \
      +TX_READY_MODE=1 +TX_READY_HIGH_CYCLES=5 \
      +TX_READY_LOW_CYCLES=8 +DRAIN_TIME_NS=3000 \
      -l sim_bp_heavy_fixed.log
```

En este escenario el receptor permanece bloqueado 8 ciclos por cada período de habilitación de 5 ciclos, lo que representa una retro-presión del 61.5 %. El plusarg +DRAIN_TIME_NS=3000 indica a la secuencia virtual que espere 3000 ns adicionales al final para que el TX FIFO termine de vaciarse antes de que la simulación concluya.

Resultado: SCB_MATCH: 12, UVM_ERROR: 0, UVM_FATAL: 0, STATUS = 0x00000000. El DUT retuvo los datos durante los períodos de bloqueo y los entregó correctamente en cuanto ready volvió a subir.

Escenario 4 — Estrés legal aleatorio sin backpressure

```
./simv +UVM_TESTNAME=aligner_legal_alignment_test \
      +UVM_VERBOSITY=UVM_LOW +FIXED_TRAFFIC=0 \
      +NUM_ITEMS=200 +TX_READY_MODE=0 \
```

```
+DRAIN_TIME_NS=3000 +ntb_random_seed=999 \
-l sim_stress_random.log
```

El driver TX mantiene `md_tx_ready=1` permanentemente (modo 0). Se generaron 200 transacciones RX aleatorias legales para estresar el pipeline con un volumen alto de datos.

Resultado: SCB_MATCH: 323, UVM_ERROR: 0, UVM_FATAL: 0, STATUS = 0x00000000. El DUT procesó correctamente las 200 transacciones y produjo 323 salidas TX alineadas, todas coincidentes con la predicción del modelo de referencia.

6.2.3 Resumen Global de Pruebas

Table 23: Resumen de todos los escenarios ejecutados

Test	Escenario	SCB	ERR	FATAL
<code>aligner_legal_alignment_test</code>	Fijo (7 ítems)	12	0	0
<code>aligner_legal_alignment_test</code>	Aleatorio (30 ítems, seed=123)	42	0	0
<code>aligner_legal_alignment_test</code>	Estrés aleatorio (200 ítems, seed=999)	323	0	0
<code>aligner_tx_backpressure_test</code>	BP periódico fijo (HIGH=2 LOW=3)	12	0	0
<code>aligner_tx_backpressure_test</code>	BP aleatorio (PCT=60, seed=321)	44	0	0
<code>aligner_tx_backpressure_test</code>	BP fuerte fijo (HIGH=5 LOW=8)	12	0	0

7 Conclusiones

La implementación del ambiente de verificación UVM para el módulo `cfs_aligner` permitió validar de forma sistemática y automatizada el comportamiento funcional del DUT bajo condiciones nominales, de estrés y de retro-presión en la interfaz de salida. A partir del trabajo realizado se extraen las siguientes conclusiones.

La separación de responsabilidades propia de UVM demostró ser fundamental para la mantenibilidad del ambiente. El hecho de que el modelo de referencia, el *scoreboard*, los agentes y las secuencias operen de forma independiente permitió modificar el comportamiento del driver TX mediante `plusargs` sin alterar ningún otro componente del ambiente.

La prueba de alineamiento legal base confirmó que el DUT procesa correctamente las siete combinaciones válidas de `CTRL.SIZE` y `CTRL.OFFSET`. En modo fijo, las 7 transacciones RX generaron exactamente los 12 bytes TX esperados, con una secuencia de salida que coincidió byte a byte con

la predicción del modelo de referencia. En modo aleatorio con 30 transacciones y semilla 123, el *scoreboard* reportó 42 matches, resultado coherente con el hecho de que transacciones con `size=2` o `size=4` generan más de un byte de salida TX por ítem RX recibido.

El escenario de estrés con 200 transacciones RX aleatorias y `ready` siempre en 1 generó 323 salidas TX correctamente alineadas sin ningún error UVM ni condición anómala en el registro STATUS. Este resultado confirma que el pipeline del DUT (RX FIFO → Controller → TX FIFO) no presenta pérdida de datos bajo volumen alto de tráfico.

Los tres escenarios de retro-presión del TX demostraron que el DUT respeta correctamente el protocolo MD en la interfaz de salida. En el modo periódico suave (`HIGH=2`, `LOW=3`) y en el modo aleatorio (60 % `ready`), el *scoreboard* confirmó todos los matches esperados. En el modo de retro-presión fuerte (`HIGH=5`, `LOW=8`), donde el receptor permanece bloqueado el 61.5 % del tiempo, el DUT retuvo los datos en el TX FIFO durante los períodos de bloqueo y los entregó correctamente en cuanto `ready` volvió a subir, validando el funcionamiento del TX FIFO como buffer de desacople.

La cobertura funcional ampliada, con los coverpoints `cp_size`, `cp_offset`, `cp_err` y `cp_data_lsb` más el cruce `size×offset`, permitió cuantificar qué combinaciones de configuración y rango de datos fueron efectivamente ejercitadas durante la simulación, complementando la verificación basada en el *scoreboard* con una métrica de exhaustividad.

El uso del modelo RAL facilitó el acceso a registros desde las secuencias, eliminando la necesidad de construir manualmente las tramas APB y permitiendo que el predictor mantenga el modelo de registros sincronizado con el estado real del DUT sin lecturas adicionales al hardware.